

Fine-Tuning For Classification and to Follow Instructions

Summary of Chapters 6-7 in “Build a Large Language Model From Scratch”
Sebastian Raschka

Fine-Tuning

- Done after Pre-Training of a model
- Considered “Post Training”
- Different goals for Fine-Tuning:
 - Instruction Fine-Tuning
 - Classification Fine-Tuning
 - Safety Alignment

Why Fine Tune?

- Right after Pre-Training a model can predict next tokens
- Predictions often result in repeated text and incoherent sentences
- Fine-tune for more desired outputs
- Ex. <https://arxiv.org/pdf/2203.02155> (OpenAI)

Prompt:

What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):  
    C = [0 for i in range(r + 1)];  
    C[0] = 1;  
    for i in range(1, n + 1):  
        j = min(i, r);  
        while j > 0:  
            C[j] += C[j - 1];  
            j -= 1;  
    return C[r]
```

GPT-3 175B completion:

- A. to store the value of C[0]
- B. to store the value of C[1]
- C. to store the value of C[i]
- D. to store the value of C[i - 1]

InstructGPT 175B completion:

The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

Before we start fine-tuning the model as a spam classifier, let's see whether the model already classifies spam messages by prompting it with instructions:

```
text_2 = (
    "Is the following text 'spam'? Answer with 'yes' or 'no':"
    " 'You are a winner you have been specially"
    " selected to receive $1000 cash or a $2000 award.'"
)
token_ids = generate_text_simple(
    model=model,
    idx=text_to_token_ids(text_2, tokenizer),
    max_new_tokens=23,
    context_size=BASE_CONFIG["context_length"]
)
print(token_ids_to_text(token_ids, tokenizer))
```

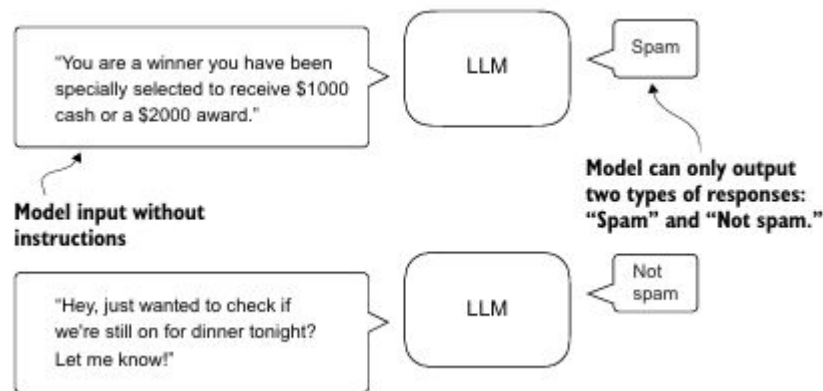
The model output is

```
Is the following text 'spam'? Answer with 'yes' or 'no': 'You are a winner
you have been specially selected to receive $1000 cash
or a $2000 award.'
The following text 'spam'? Answer with 'yes' or 'no': 'You are a winner
```

Based on the output, it's apparent that the model is struggling to follow instructions. This result is expected, as it has only undergone pretraining and lacks instruction fine-tuning. So, let's prepare the model for classification fine-tuning.

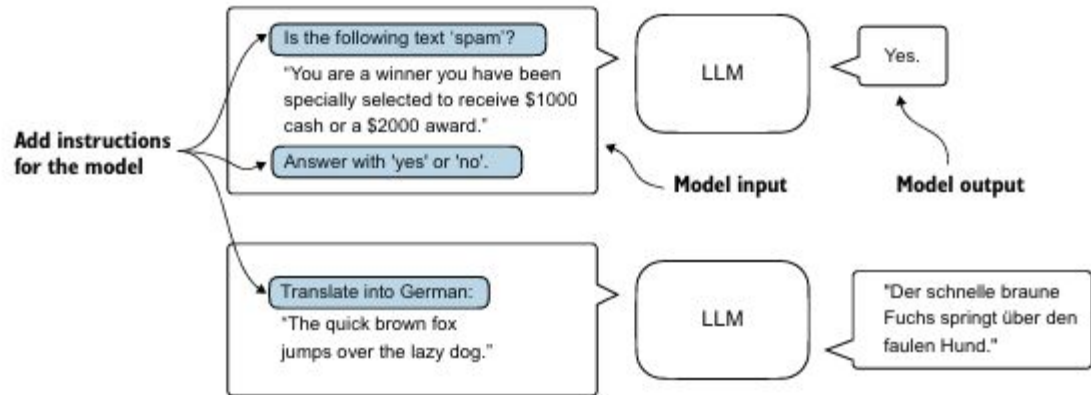
Classification Fine-Tuning

- Tuning model to perform some type of classification
- Ex. Classification Tasks:
 - Spam vs Not-Spam
 - Sentiment Classification (Happy vs Sad)
 - Topic Classification
 - Fake News Detection
 - Identifying object class visually
- Similar to more traditional ML tasks
- Goal is to Restrict the Model



Instruction Fine-Tuning

- Tuning model to better follow instructions, and execute tasks provided as prompts from the user
- Ex. of an “Instruction” or “Task”
 - Summarization
 - Question answering
 - Translation
 - Rewriting
 - Content Extraction
 - Planning
- More Generalized Model



Choosing Instruction or Classification

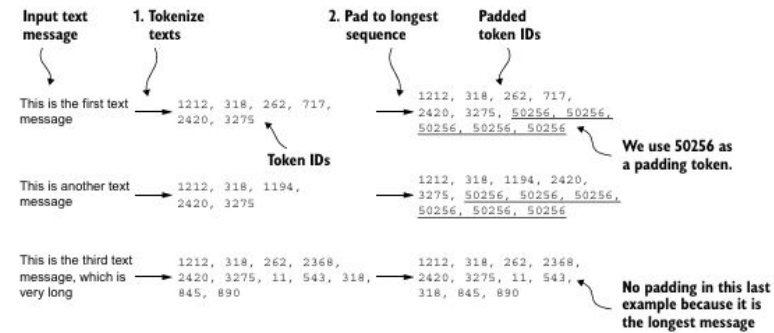
- Choose Instruction-Tuning for a more generalized model in aims to respond to a wide variety of tasks and prompts
- Instruction-Tuning requires larger datasets for tuning
- Choose Classification-tuning if the scope of the model is narrower, e.g. the model should perform one specific tasks with a closed domain of classes

Implementing Classification FT 1 - Dataset Preparation

- The Dataset
 - Includes pairs of object/text to be classified -> classification label
 - Download the dataset
- Preprocessing the Dataset
 - Inspect the dataset to make sure it the class labels are balanced
 - Convert class labels to integer class labels or one-hot vector encodings
 - Potentially perform text cleaning, remove excessive whitespace, remove html tags etc
 - Handle NA values, missing values, and duplicate instances
- Dataset Split
 - Split the dataset for training, validation and testing (validation used for tuning hyperparameters)
 - Textbook recommends a 70-10-20 split

Implementing Classification FT 1.1 - Dataset Preparation

- Data loaders handle feeding the model the data in the right shape, format and order
- The data loader separates the dataset in batches, to better make use of gpu performance
- Abstract Preprocessing:
 - Truncating or padding inputs (each input has to be the same size)
 - Padding token is a special token (in the textbook `<|endoftext|>` is used)
 - Can add `<|endoftext|>` token id to the encoded inputs to save compute

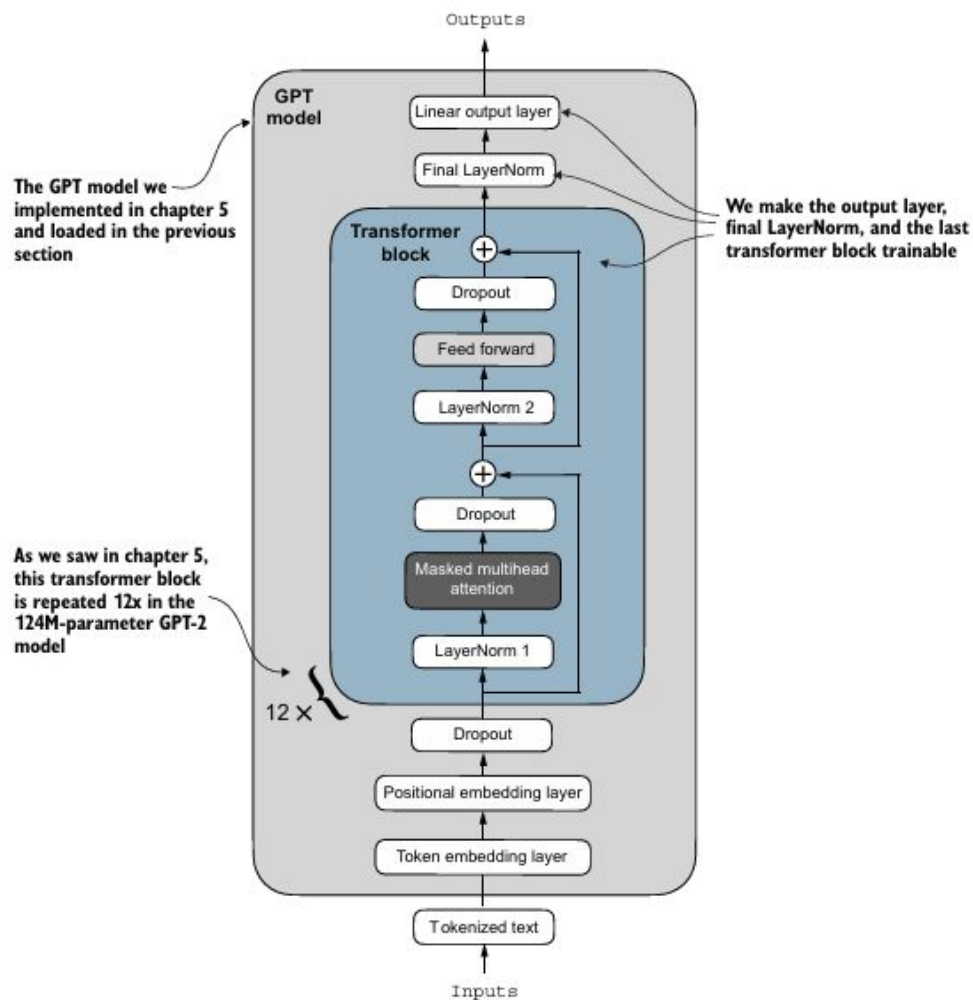


Implementing Classification FT 2 Model Setup

- Initializing a Model w/ Pretrained Weights, Textbook goes through self-implemented model, but will be different for hugging face model
- Load pretrained weights -> `AutoModelForCausalLM.from_pretrained()`
- We have to add a classification head:
 - Replace output layer from `#nodes = len(token_vocab)` to `#nodes = len(class_labels)`
 - Now each logit in the output layer will represent the probability of the corresponding class label
 - Next best token prediction -> Class label prediction
 - Perform fine tuning on the last output layer

Implementing Classification FT 2.1 Model Setup

- Fine-tuning all layers vs selected layers:
 - If model is pretrained, it is not necessary to fine-tune all model layers
 - Lower layers capture basic language structure and semantics
 - Fine-tuning only the last layer is often sufficient (especially for a relatively easy NLP task such as Spam detection)
- For tuning, we freeze all layers besides the output layer and last transformer block
- Increasing the # of tuned Layers can improve performance



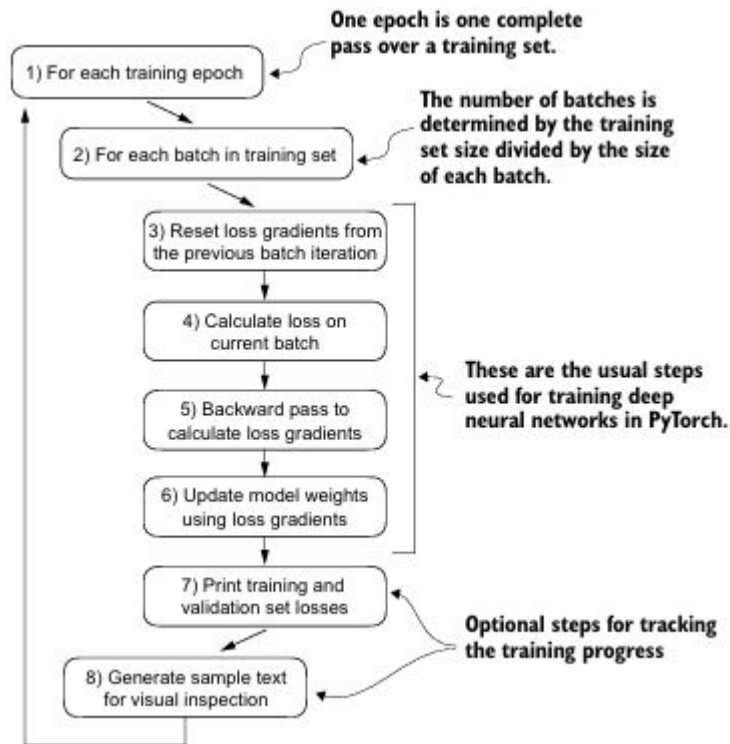
Implementing Classification FT 2.2 Evaluation Utilities

- Classification accuracy can be calculated by # of Correct Classifications / # of Classification Instances
- To Fine-Tune a Loss Function Must be defined to be optimized
- For classification we can use a cross-entropy loss function
- Want to update weights to minimize the loss

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Implementing Classification FT 3 Fine-Tuning

Follows this general outline:



Implementing Instruction FT 1 - Dataset Preparation

- Dataset Preparation is similar to fine-tuning for classification outside of the formatting of the dataset
- For instruction tuning datasets each tuple in the dataset should follow the general format; instruction, input, and output
- These tuples may be extracted from JSON, similar to some of our initial formatings
- There are different commonly used templates (prompt styles) for instruction tuning (ex. Alpaca, phi-3)

Example entry:

```
{ 'instruction': 'Identify the correct spelling of the following word.',  
  'input': 'Ocassion', 'output': "The correct spelling is 'Occasion.'" }
```

Implementing Instruction FT 1.1 - Dataset Preparation

- To do the dataset formatting, define a function that takes in a dictionary/json and returns the formatted instruction, input, output pair
- The same idea of dataset splitting in classification FT occur in instruction FT
- For the data loader:
 - Each tuple is first converted to token ID's, padded with end-of-text tokens (similar to classification)
 - Then a target token ID's have to be generated, these are the tokens that we want the model to learn for the given input as well as what is used for the loss function
 - Target token ID's match the input token ID's, shifted to the left 1 (remove the first input token ID) and only padded with one end-of-text token ID then padded with a certain value to represent exclusion of the training loss calculation (-100, pytorch cross Entropy function ignores this value)

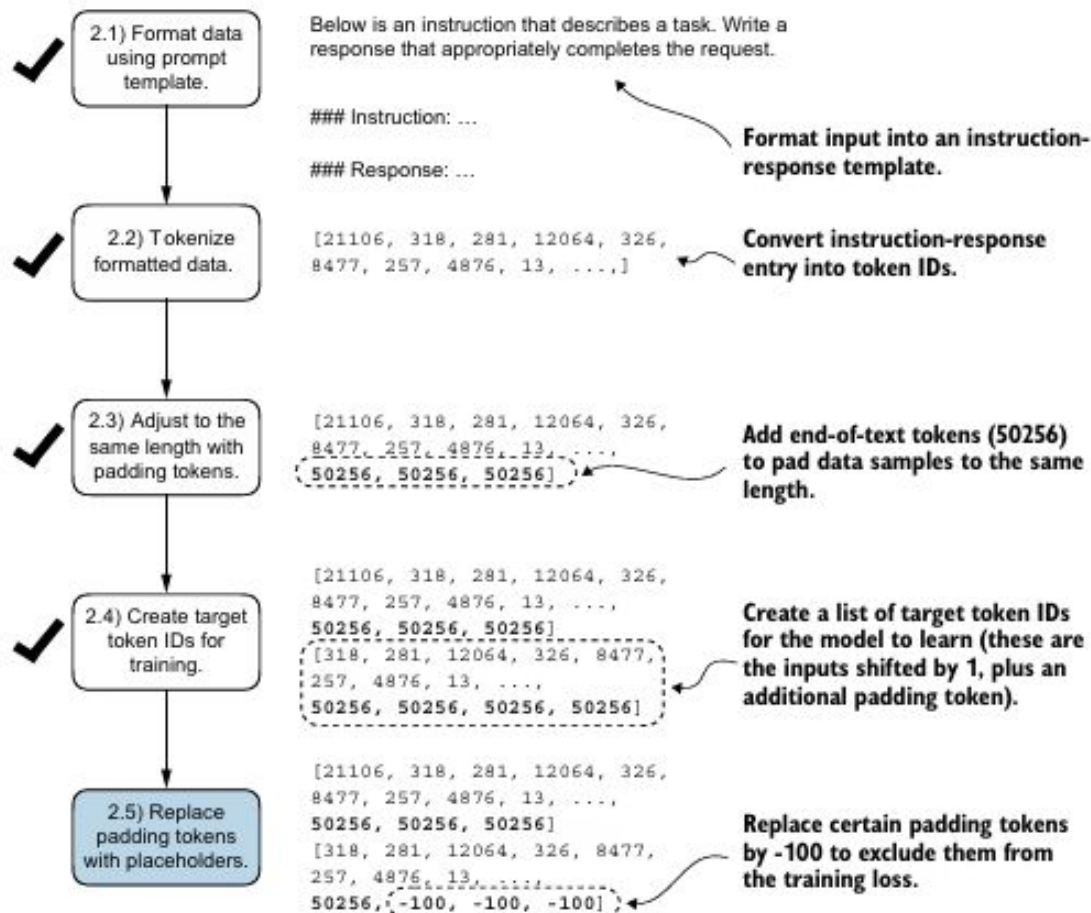


Figure 7.11 The five substeps involved in implementing the batching process. After creating the target sequence by shifting token IDs one position to the right and appending an end-of-text token, in step 2.5, we replace the end-of-text padding tokens with a placeholder value (-100).

Implementing Instruction FT 1.2 - Dataset Preparation

- It is also very common to mask the instruction and input tokens in the target tokens with -100, so that the cross entropy loss is only computed on the response tokens
- Promotes training on generating responses rather than instructions and responses

Input text:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.

↓ Tokenize

[21106, 318, 281, 12064, 326, ..., 13]

↖
The token IDs corresponding
to the input text

mask out the instruction
when calculating the loss.

Target text:

is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:
Rewrite the following sentence using passive voice.

Input:
The team achieved great results.

Response:
Great results were achieved by the team.<|endoftext|>

↓ Tokenize

[-100, -100, -100, -100, -100, ..., 13, 50256]

↖
The instruction tokens
are replaced by -100.

Implementing Instruction FT 2 - Fine Tuning

- We will load the pretrained model and weights
- Much of the fine-tuning is now very similar to when pretraining
- We use the same loss calculations and training functions used for pre-training
- Computes Cross-Entropy Loss across all target tokens and averages or sums them
- The gradient is computed and weights are updated to minimize the loss in a backward pass (backpropagation)